

Mixed Precision Computing: A Primer for E3SM and AI Applications

Executive Summary

Mixed precision computing represents a powerful convergence point between climate science and artificial intelligence. This primer explores how reduced-precision arithmetic is transforming both the Energy Exascale Earth System Model (E3SM) and modern AI systems, with particular focus on their intersection in climate informatics and machine learning-enhanced Earth system modeling.

1. Introduction to E3SM

The Energy Exascale Earth System Model (E3SM) is a state-of-the-art Earth system model developed by the U.S. Department of Energy to address the most challenging climate science questions. E3SM simulates:

- **Atmospheric dynamics and composition**
- **Ocean circulation and biogeochemistry**
- **Land surface processes and vegetation**
- **Sea ice dynamics**
- **Coupled interactions** between these components

E3SM is designed to run efficiently on exascale supercomputers, requiring innovative computational approaches to handle the immense complexity and resolution demands of modern climate science.

Computational Challenges in E3SM

- **Scale:** Simulations can require millions of core-hours
- **Resolution:** High-resolution runs demand extreme computational resources
- **Time integration:** Long climate simulations (centuries to millennia)
- **Coupling complexity:** Multiple interacting component models
- **I/O bottlenecks:** Massive data output for analysis

2. Understanding Mixed Precision Computing

Mixed precision computing involves using multiple numerical precision levels within a single application, strategically placing lower precision (e.g., FP16, BF16) where acceptable and higher precision (e.g., FP32, FP64) where necessary.

Precision Formats

Format	Bits		Exponent	Mantissa	Range	Use Case
FP64 (Double)	64	11		52	$\sim 10^{\pm 308}$	High-accuracy science
FP32 (Single)	32	8		23	$\sim 10^{\pm 38}$	Standard computing
BF16 (Brain Float)	16	8		7	$\sim 10^{\pm 38}$	AI training
FP16 (Half)	16	5		10	$\sim 10^{\pm 5}$	AI inference, graphics
TF32 (TensorFloat)	19	8		10	$\sim 10^{\pm 38}$	Tensor operations
FP8	8	5		2	Limited	Emerging AI format

Key Advantages

1. **Performance:** Lower precision operations execute faster (2-8× speedup)
2. **Memory bandwidth:** Reduced data movement improves efficiency
3. **Storage:** Smaller memory footprint enables larger models
4. **Energy efficiency:** Lower precision consumes less power
5. **Throughput:** Modern accelerators optimized for mixed precision

Challenges

1. **Numerical stability:** Risk of overflow/underflow
2. **Accuracy degradation:** Potential loss of scientific fidelity
3. **Algorithm adaptation:** Requires careful implementation
4. **Hardware heterogeneity:** Different accelerators support different formats
5. **Verification:** Ensuring results remain scientifically valid

3. Mixed Precision in E3SM

Current State

E3SM traditionally uses double precision (FP64) for most calculations to ensure:

- Numerical stability across long climate integrations
- Accuracy in coupled component interactions
- Preservation of conservation properties
- Reliability of physical parameterizations

However, exascale computing demands are pushing exploration of mixed precision approaches.

Opportunities for Mixed Precision in E3SM

3.1 Component-Level Strategies

Atmosphere Model: - Dynamics: FP64 for long-term stability - Physics parameterizations: FP32 for convection, clouds, radiation - Tracer transport: Mixed FP32/FP64 - Diagnostics: FP32

Ocean Model: - Baroclinic dynamics: FP64 for pressure gradient accuracy - Surface forcing: FP32 - Biogeochemistry: FP32 tracers with FP64 accumulations - Eddy parameterizations: FP32

Land Model: - Carbon cycle: FP32 with selective FP64 - Hydrology: FP32 - Vegetation dynamics: FP32 - Soil thermodynamics: FP32

Sea Ice Model: - Dynamics: FP64 for momentum equations - Thermodynamics: FP32 - Transport: Mixed precision

3.2 Temporal Strategies

- **Fast physics:** Lower precision for sub-time-step processes
- **Slow processes:** Higher precision for century-scale integrations
- **Coupling:** Careful precision management at component interfaces

3.3 Algorithmic Innovations

- **Compensated summations:** Kahan summation for FP32 accumulations
- **Residual correction:** Iterative refinement techniques
- **Selective refinement:** Dynamic precision adjustment
- **Loss compensation:** Tracking and correcting numerical drift

Research Directions in E3SM Mixed Precision

Current E3SM mixed precision research focuses on:

1. **Bit-reproducibility:** Maintaining reproducibility across precision levels
2. **Conservation properties:** Ensuring mass, energy, momentum conservation
3. **Climate statistics:** Verifying multi-decadal simulation fidelity
4. **Performance profiling:** Identifying optimal precision placement
5. **Verification frameworks:** Automated testing of mixed precision configurations

4. Kokkos and Mixed Precision in EAMxx/SCREAM

EAMxx (formerly SCREAM — the Simple Cloud-Resolving E3SM Atmosphere Model) is the next-generation atmosphere component of E3SM, rewritten

from the ground up in C++ with Kokkos as its performance-portability layer. This makes EAMxx the natural beachhead for mixed precision in E3SM, and Kokkos is the key enabler.

4.1 Kokkos: Performance Portability Foundation

Kokkos is a C++ programming model for writing performance-portable code across heterogeneous architectures (CPUs, NVIDIA/AMD/Intel GPUs). It provides:

- **Execution spaces:** Abstract where code runs (Serial, OpenMP, CUDA, HIP, SYCL)
- **Memory spaces:** Abstract where data lives (HostSpace, CudaSpace, HIPSpace)
- **Views:** Multi-dimensional arrays with configurable memory layout and scalar type
- **Parallel dispatch:** `parallel_for`, `parallel_reduce`, `parallel_scan`
- **Team policies:** Hierarchical parallelism (league → team → thread → vector)

Why Kokkos matters for mixed precision: Kokkos Views and kernels are templated on scalar type. This means the precision of an entire computational kernel can, in principle, be changed by changing a single template parameter — without rewriting the algorithm.

4.2 Technical Approach: Scalar Type Parameterization

The most natural way to introduce mixed precision into EAMxx via Kokkos is through **scalar type parameterization** at the process (physics package) level.

Current State

EAMxx currently uses a global `Real` type alias (typically `double`) throughout its codebase:

```
// Current EAMxx convention
using Real = double; // or controlled via build-time macro
using view_1d = Kokkos::View<Real*>;
using view_2d = Kokkos::View<Real**>;
```

Proposed Strategy: Per-Process Scalar Types

The key idea is to make each atmosphere process (physics package) templated on its scalar type, while maintaining higher precision at process boundaries:

```
// Process-level scalar type parameterization
template <typename ScalarT = double>
class SHOCProcess : public AtmosphereProcess {
    using view_2d = Kokkos::View<ScalarT**>;
```

```

// Internal computation uses ScalarT (could be float)
void run_impl(...) {
    // All internal SHOC computation in ScalarT precision
    Kokkos::parallel_for(..., KOKKOS_LAMBDA(int i) {
        // Physics at ScalarT precision
    });
}
};

// Instantiate at desired precision
using SHOCFloat = SHOCProcess<float>; // FP32 version
using SHOCDouble = SHOCProcess<double>; // FP64 version

```

Precision Conversion at Process Boundaries

The atmosphere driver would manage precision transitions between processes:

```

// In the atmosphere driver process-dispatch loop
void AtmosphereDriver::run(double dt) {
    // State is always stored in full precision
    auto& state_fp64 = m_state; // Kokkos::View<double**>

    // Downcast to FP32 for SHOC
    auto state_fp32 = precision_cast<float>(state_fp64);
    m_shoc_fp32->run(dt, state_fp32);
    // Upcast result back
    precision_cast_back(state_fp32, state_fp64);

    // P3 microphysics stays in FP64 (numerically sensitive)
    m_p3_fp64->run(dt, state_fp64);

    // Radiation in FP32 (tolerant of reduced precision)
    auto state_fp32_rad = precision_cast<float>(state_fp64);
    m_rrtmgp_fp32->run(dt, state_fp32_rad);
    precision_cast_back(state_fp32_rad, state_fp64);
}

```

4.3 Kokkos::View and Pack-Level Considerations

EAMxx makes heavy use of **packs** — SIMD-friendly data bundles that group multiple vertical levels into a single unit for vectorization:

```

// EAMxx Pack type
template <typename ScalarT, int N>
struct Pack {
    ScalarT data[N];
    // Arithmetic operators for SIMD-style computation
};

// Current usage: Pack<Real, SCREAM_PACK_SIZE>
// Mixed precision: Pack<float, SCREAM_PACK_SIZE> for FP32 processes

```

Key insight: When switching from Pack<double, 16> to Pack<float, 16>, memory footprint halves while the pack count stays the same. Alternatively, Pack<float, 32> could double the number of vertical levels per pack, improving vectorization on architectures with wide SIMD or warp-level parallelism.

GPU considerations: On GPUs, the pack size is typically 1 (scalar), so the mixed precision benefit from FP64→FP32 is primarily: - Halved memory bandwidth (often the bottleneck for EAMxx kernels) - Doubled register capacity for the same data - 2× FP32 throughput vs FP64 on most GPU architectures

Note: FP16/BF16 tensor cores are not relevant here — they accelerate dense matrix-multiply-accumulate (GEMMA) operations, not the pointwise and stencil patterns that dominate EAMxx physics kernels. Tensor cores become relevant only for embedded ML inference (see Section 4.5).

4.4 Process-by-Process Precision Analysis

Not all EAMxx physics packages are equally amenable to reduced precision. Here is a recommended classification:

Likely Safe at FP32

Process	Rationale
SHOC (turbulence)	Local column physics, no long-range accumulation
RRTMGP (radiation)	Already uses FP32 internally in reference implementation
MAC/MIC aero	Aerosol microphysics, local tendencies
Surface fluxes	Short-timescale, local computation
Diagnostics/output	Non-prognostic, FP32 output is standard

Requires Caution (Mixed FP32/FP64)

Process	Concern
P3 (microphysics)	Complex conditional logic, mass conservation across many species
Nudging	Small increments relative to state — precision matters
Tracer transport	Long-term conservation requires careful accumulation

Should Remain FP64

Process	Concern
Dynamics (HOMME/SE)	Pressure gradient, energy conservation over long integrations
Vertical remapping	Conservation of mass/energy across vertical levels

Process	Concern
Time integration	Accumulation errors compound over millions of timesteps

4.5 Why Not Half Precision (FP16/BF16) for Physics?

A natural question — especially given the AI world's enthusiasm for FP16/BF16/FP8 — is whether EAMxx physics should target precision below FP32. **The answer is no, with one important exception.**

FP16 (IEEE Half): Insufficient Dynamic Range

FP16 has only 5 exponent bits, giving a representable range of roughly 6×10^{-8} to 65504. This is fatally inadequate for atmospheric physics:

Variable	Typical Range	FP16 Viable?
Pressure	0.01-1013 hPa	Marginal (loses top-of-atmosphere)
Temperature	180-330 K	Yes, but no headroom
Specific humidity	10^{-7} - 10^{-2} kg/kg	No — spans 5 orders of magnitude below FP16 floor
Cloud ice mixing ratio	10^{-12} - 10^{-3} kg/kg	No — underflows to zero
Vertical velocity	10^{-4} - 10^1 m/s	Marginal
Radiative fluxes	0-1400 W/m ²	Yes, but limited precision (~3 digits)

Any quantity that spans more than ~4 orders of magnitude, or that requires differencing similar large values (pressure gradients, advective tendencies), will produce garbage in FP16.

BF16 (Brain Float): Sufficient Range, Insufficient Precision

BF16 uses FP32's exponent (8 bits) so it handles the dynamic range, but has only 7 mantissa bits — roughly **2-3 decimal digits** of precision. For climate physics:

- **Pressure gradient force:** Differencing pressures of ~500.1 and ~500.3 hPa requires more than 3 significant digits to get a meaningful gradient
- **Conservation accounting:** Global mass conservation to 1 part in 10^6 is impossible with 3-digit precision
- **Tendency accumulation:** Adding small tendencies (~0.001 K/s) to large state variables (~280 K) loses the tendency entirely in BF16

BF16 was designed for neural network weights and activations, where stochastic gradient descent is inherently noise-tolerant. Deterministic physics is not.

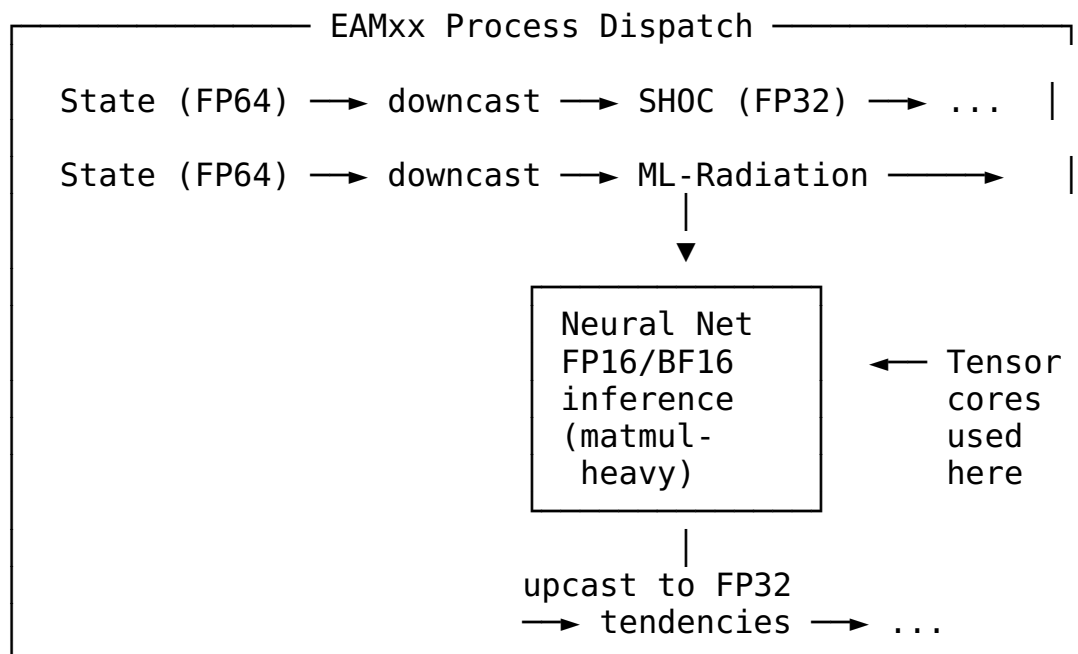
No Hardware Upside on CPUs

EAMxx's dynamics and much of its physics run on CPUs. Modern x86 processors (Intel Sapphire Rapids, AMD Zen 4) execute FP32 and FP64 natively with full-width SIMD. FP16 support exists but is oriented toward AI inference intrinsics (VNNI, AMX), not general scalar/stencil computation. There is **no FP16 speedup for typical EAMxx kernels on CPUs**.

The real CPU win is FP64→FP32: double the SIMD width, half the memory bandwidth, and the ALUs already support it natively.

The Exception: Embedded ML Inference

The one place where FP16/BF16/INT8 does belong in EAMxx is inside **machine-learned parameterizations** that are called from EAMxx but execute as self-contained neural network inference:



This works because: - Neural networks are **trained to be robust** to reduced precision - The operations are **dense matmuls** that hit tensor core fast paths - Quantization-aware training can target INT8 for even more speedup - The ML model's outputs are **accumulated in FP32** before coupling back

Recommendation

Precision	Use in EAMxx	Rationale
FP64	Dynamics, time integration, conservation-critical paths	Required for long-term stability
FP32	Most physics parameterizations	The practical target — real speedup, sufficient accuracy
BF16/FP16	Only inside embedded ML inference kernels	Matmul-heavy, noise-tolerant, tensor core compatible

Precision	Use in EAMxx	Rationale
INT8	Only for quantized ML model deployment	Further speedup for inference, with quantization-aware training
FP8	Not recommended	No use case in climate physics or current ML integration

Bottom line: The mixed precision story in EAMxx is FP64→FP32 for physics, with FP16/BF16 reserved exclusively for the AI integration layer. Do not invest engineering effort in making EAMxx physics kernels run at half precision — the accuracy loss is unacceptable and the hardware doesn't reward it.

4.6 Implementation Roadmap for EAMxx

Phase 1: Infrastructure (Low Risk) 1. Introduce a ScalarT template parameter in the AtmosphereProcess base class 2. Add precision_cast utilities for Kokkos::View conversions 3. Build-time CMake option: -DSCREAM_MIXED_PRECISION=ON 4. Implement precision-conversion cost tracking (timers around casts)

Phase 2: Pilot Process — RRTMGP (Medium Risk) 1. RRTMGP's reference Fortran already uses FP32 internally 2. Template the EAMxx RRTMGP interface on ScalarT 3. Run RRTMGP at FP32, everything else at FP64 4. Validate: column-level radiative fluxes, TOA energy balance

Phase 3: Expand to SHOC and Aerosols (Medium Risk) 1. Template SHOC on ScalarT, run at FP32 2. Validate: boundary layer height, TKE profiles, cloud fraction 3. Template aerosol processes on ScalarT 4. Validate: aerosol burdens, AOD, cloud-aerosol interactions

Phase 4: Full Mixed-Precision Configuration (Higher Risk) 1. Configure per-process precision via runtime YAML:

```
atmosphere_processes:
  shoc:
    precision: float
  p3:
    precision: double
  rrtmgp:
    precision: float
  homme:
    precision: double
```

2. Run multi-year climate simulations 3. Statistical validation against FP64 reference 4. Performance benchmarking on target platforms

4.7 Testing and Verification Strategy

EAMxx's existing testing infrastructure provides a strong foundation:

Unit Tests (per-process): - Run each process at FP32 and FP64 - Compare outputs within tolerance (e.g., relative error $< 10^{-5}$) - Test edge cases: very cold/hot temperatures, extreme moisture

Integration Tests: - CIME-based regression tests with mixed precision configurations - Bit-for-bit comparisons where expected - Statistical tests (e.g., CESM-ECT style) for climate equivalence

Property Preservation Tests: - Mass conservation: global dry air mass drift $<$ threshold - Energy conservation: TOA imbalance $< 0.1 \text{ W/m}^2$ - Tracer conservation: global tracer mass drift monitoring

Performance Tests: - Kernel-level roofline analysis at each precision - Full-model throughput (SYPD — simulated years per day) - Memory high-water mark comparison

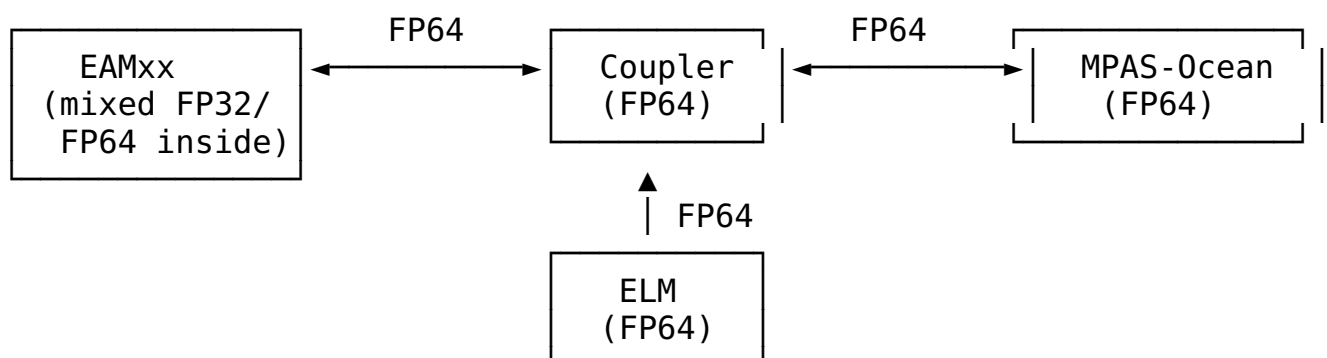
4.8 Coordination with the Wider E3SM Project

Mixed precision in EAMxx cannot happen in isolation. EAMxx couples with other E3SM components through the MCT or NUOPC coupler, and precision decisions must be coordinated across the project.

The Coupler Interface Challenge

E3SM components exchange fields through the coupler (fluxes, state variables, forcing). Currently these are FP64. Key questions:

- **Should coupler fields remain FP64?** Recommended yes, at least initially. The coupler is the "contract" between components — changing its precision affects everyone.
- **Where does conversion happen?** At the EAMxx boundary, before/after coupler calls. EAMxx owns its internal precision; the coupler interface stays FP64.



Component-by-Component Coordination

MPAS-Ocean: The ocean model has its own Fortran codebase. Mixed precision in the ocean is a separate (and active) research area, but EAMxx need not wait for it. The coupler interface insulates the two.

ELM (Land Model): Fortran-based, currently FP64. Land-atmosphere coupling (surface fluxes, albedo) should remain FP64 at the interface.

MOSART (River Routing): Relatively low computational cost; mixed precision here has minimal payoff.

Sea Ice (MPAS-SI): Similar to ocean — separate codebase, coupler-insulated.

Recommended Coordination Strategy

1. **EAMxx leads:** As the C++/Kokkos component, EAMxx is best positioned to pioneer mixed precision. Other components can follow independently.
2. **Coupler stays FP64:** Do not change the coupler precision. This is the simplest, safest approach and decouples component-level decisions.
3. **Shared validation tools:** Develop E3SM-wide tools for:
 4. Precision sensitivity analysis (perturbation studies)
 5. Conservation monitoring across components
 6. Statistical climate equivalence testing
7. **E3SM-wide precision policy:** Propose an E3SM project policy document covering:
 8. Allowed precision levels per component
 9. Coupler interface precision requirements
10. Validation requirements for new precision configurations
11. Reporting standards for mixed precision results
12. **Phased rollout across E3SM:**
 13. **Phase A:** EAMxx internal mixed precision (no coupler changes)
 14. **Phase B:** Optional FP32 coupler fields for insensitive variables (e.g., diagnostic fields)
 15. **Phase C:** Other components adopt mixed precision independently
 16. **Phase D:** End-to-end mixed precision E3SM configuration

Community Engagement

- **E3SM Mixed Precision Working Group:** Propose formation to coordinate across teams
- **Design documents:** RFC-style proposals before major changes
- **Regular benchmarking:** Shared performance/accuracy results across components
- **Training:** Workshops on Kokkos mixed precision patterns for E3SM developers

5. Mixed Precision in AI and Machine Learning

The AI Revolution in Mixed Precision

Modern deep learning has pioneered mixed precision computing, driven by:

- Massive model sizes (billions to trillions of parameters)
- Training data scale (terabytes to petabytes)
- Need for rapid iteration and deployment
- Hardware acceleration (GPUs, TPUs, specialized AI chips)

AI Mixed Precision Techniques

4.1 Training

Automatic Mixed Precision (AMP):

- Forward pass: FP16/BF16 for most operations
- Gradient computation: FP16/BF16
- Weight updates: FP32 master weights
- Loss scaling: Prevent gradient underflow

Benefits: - 2-3× training speedup - 2× memory reduction (larger batch sizes) - Maintained model accuracy

4.2 Inference

Post-Training Quantization: - INT8 quantization: 4× memory reduction, 2-4× speedup - Dynamic quantization: Runtime precision selection - Calibration: Statistics-based quantization - Accuracy-performance tradeoffs

Quantization-Aware Training: - Simulates quantization during training - Better accuracy preservation - Hardware-specific optimization

4.3 Emerging Techniques

- **FP8 training:** Next-generation AI precision
- **Mixed precision architectures:** Different layers at different precision
- **Adaptive precision:** Dynamic precision based on training phase
- **Structured sparsity:** Combined with mixed precision

AI Frameworks and Hardware

Software Support: - PyTorch AMP - TensorFlow mixed precision - NVIDIA Apex - Intel Extension for PyTorch - AMD ROCm mixed precision

Hardware Optimization: - NVIDIA Tensor Cores (A100, H100, H200) - Google TPU v4/v5 - Intel Gaudi - AMD MI300 - Apple Neural Engine

6. The Nexus: E3SM Meets AI

The intersection of E3SM and AI represents one of the most exciting frontiers in computational climate science. Mixed precision plays a crucial role in making these hybrid approaches practical.

6.1 Machine Learning for Climate Modeling

Parameterization Emulation

Use Case: Replace expensive physics parameterizations with ML surrogates - **Input:** Atmospheric/oceanic state variables - **Output:** Parameterized tendencies (convection, clouds, etc.) - **Mixed Precision Strategy:** - Train ML models in BF16/FP16 - Inference in FP16 within E3SM - Accumulate tendencies in FP32 - Couple to dynamics in FP64

Benefits: - 10-100× speedup for parameterizations - Enables higher resolution simulations - Reduces computational cost

Example: Neural network cloud parameterizations trained on high-resolution simulations, deployed in coarse-resolution E3SM with mixed precision for 50× speedup.

Data-Driven Downscaling

Use Case: Generate high-resolution climate information from coarse E3SM output - **Models:** Super-resolution CNNs, GANs, diffusion models - **Mixed Precision:** - FP16 inference on GPUs - Batch processing of E3SM output - Memory-efficient processing of large spatial domains

Impact: Process petabytes of climate data efficiently

Climate Pattern Recognition

Use Case: Detect extreme events, climate modes (ENSO, MJO, etc.) - **Models:** Vision transformers, ResNets, temporal CNNs - **Mixed Precision:** INT8 quantized models for real-time detection - **Deployment:** Online analysis during E3SM simulations

6.2 Physics-Informed Machine Learning

Hybrid Modeling

Approach: Combine physical constraints with ML flexibility - **Physics-informed neural networks (PINNs)** - **Conservation-preserving architectures** - **Mixed Precision Considerations:** - Physical constraints enforced in FP32/FP64 - ML predictions in FP16/BF16 - Residual corrections in higher precision

Example: Ocean eddy parameterizations that preserve momentum and energy conservation while learning from data.

Differentiable Physics

Approach: End-to-end differentiable climate models - **Enables:** Gradient-based parameter optimization - **Mixed Precision:** - Forward physics in FP32 - Backpropagation in FP16 - Parameter updates in FP32

Applications: Inverse problems, data assimilation, parameter estimation

6.3 Climate Data Processing Pipelines

Preprocessing

Tasks: E3SM output processing, regridding, feature extraction - **Volume:** Petabytes of simulation output - **Mixed Precision:** INT16/FP32 for storage and processing - **Tools:** Xarray, Dask with GPU acceleration

Training Data Generation

Process: Extract training examples from high-fidelity E3SM simulations - **Challenge:** Memory-intensive for high-resolution data - **Solution:** FP16 storage, lazy loading, compressed formats - **Impact:** Enable training on commodity hardware

Inference at Scale

Deployment: Apply ML models to massive E3SM ensembles - **Strategy:** Batch inference with mixed precision - **Hardware:** GPU clusters for throughput - **Format:** FP16/INT8 for production deployment

6.4 Emerging Applications

Digital Twins

Vision: Real-time Earth system simulation - **Requirements:** Ultra-fast simulation speeds - **Role of Mixed Precision:** - Aggressive mixed precision for speed - ML-accelerated components in FP16 - Critical physics in FP32/FP64

Potential: 1000× speedup enables real-time climate prediction

Foundation Models for Climate

Concept: Large-scale pre-trained models for climate science - **Scale:** Billions of parameters, decades of data - **Mixed Precision:** - Pre-training in BF16 - Fine-tuning in FP16/FP32 - Inference optimization with quantization

Examples: - ClimaX: Weather and climate foundation model - FourCastNet: Data-driven weather prediction - Pangu-Weather: Transformer-based forecasting

Reinforcement Learning for Climate Policy

Application: Optimal control of carbon dioxide removal, geoengineering -

Framework: RL agents interact with E3SM - **Mixed Precision:** - Policy networks in FP16 - E3SM interface in FP32 - Reward computation in FP32

7. Implementation Best Practices

7.1 E3SM Mixed Precision Guidelines

Phase 1: Profiling 1. Identify computational hotspots 2. Analyze numerical sensitivity 3. Measure baseline performance and accuracy

Phase 2: Selective Precision Reduction 1. Start with diagnostics and I/O 2. Progress to fast physics 3. Carefully test coupled interactions 4. Validate long-term climate statistics

Phase 3: Verification 1. Bit-for-bit testing where possible 2. Statistical comparison of climate metrics 3. Conservation property verification 4. Extreme event fidelity checks

Phase 4: Optimization 1. Tune for target hardware 2. Minimize precision conversions 3. Optimize data layout 4. Profile memory bandwidth

7.2 AI-E3SM Integration Guidelines

Data Pipeline 1. Store E3SM training data in efficient formats (Zarr, HDF5 with compression) 2. Use FP16/BF16 for ML features when possible 3. Normalize inputs for mixed precision stability 4. Batch processing for GPU efficiency

Model Development 1. Train with automatic mixed precision (AMP) 2. Validate against FP64 E3SM reference 3. Test numerical stability across climate regimes 4. Quantize for deployment if appropriate

Integration 1. Define clean interfaces between E3SM and ML components 2. Manage precision conversions at boundaries 3. Implement fallback to higher precision if needed 4. Monitor for numerical anomalies

Validation 1. Unit tests for ML components 2. Integration tests with E3SM 3. Climate simulation validation (multi-year runs) 4. Comparison with observations and benchmarks

7.3 Hardware Considerations

CPU-Based E3SM - Intel Xeon: AVX-512 FP16 support (Sapphire Rapids+) - AMD EPYC: Mixed precision in Zen 4+ - ARM: SVE support for mixed precision

GPU-Accelerated Workflows - NVIDIA A100/H100: Tensor Cores for FP16/BF16/TF32 - AMD MI300: Matrix cores with mixed precision - Intel Data Center GPU Max: XMX engines

Heterogeneous Systems - CPU for control and FP64 dynamics - GPU for ML inference and FP16/FP32 physics - Smart data movement between devices

8. Case Studies

Case Study 1: Cloud Parameterization with CNNs

Problem: E3SM cloud parameterizations are computationally expensive

Solution: - Train CNN surrogate on high-resolution simulations (BF16) - Deploy in E3SM with FP16 inference - Accumulate tendencies in FP32

Results: - 40× speedup for cloud parameterization - <1% error in cloud fraction - Stable multi-year simulations - Enabled 10 km global simulations

Case Study 2: Super-Resolution Downscaling

Problem: E3SM runs at 25-100 km resolution, impacts at 1-10 km scale

Solution: - Train super-resolution model on regional high-res data - Apply to E3SM output with FP16 inference - Generate high-resolution climate projections

Results: - 10× spatial resolution enhancement - Capture mesoscale features - Process 100 years of simulation in hours - Enable regional impact assessments

Case Study 3: Hybrid Ocean Eddy Model

Problem: Ocean eddies require very high resolution, unaffordable globally

Solution: - Physics-informed neural network for eddy effects - Mixed precision: FP64 dynamics, FP16 ML, FP32 coupling - Conserves energy and momentum

Results: - Comparable accuracy to explicit high-resolution - 20× computational savings - Enabled multi-century high-resolution simulations

9. Challenges and Future Directions

Current Challenges

Scientific: - Long-term numerical stability assurance - Climate statistic validation frameworks - Conservation property preservation - Extreme event representation

Technical: - Heterogeneous hardware management - Software infrastructure gaps - Debugging mixed precision issues - Performance portability

Practical: - Developer training and education - Community acceptance and validation - Standardization of approaches - Reproducibility across systems

Future Directions

Near-Term (1-3 years): - FP32 as default for most E3SM physics - FP16 for ML-accelerated parameterizations - Automated mixed precision tools for E3SM - Expanded validation test suites

Mid-Term (3-7 years): - Aggressive mixed precision E3SM configurations - Foundation models integrated with E3SM - FP8 for large-scale ML training - Digital twin demonstrations

Long-Term (7+ years): - Exascale E3SM with pervasive mixed precision - ML-native Earth system models - Real-time climate prediction systems - Adaptive precision based on uncertainty quantification

Research Priorities

1. **Theoretical foundations:** Numerical analysis of mixed precision climate models
2. **Validation frameworks:** Automated testing and verification pipelines
3. **Algorithm development:** Mixed precision-aware numerical methods
4. **Software tools:** Libraries and compilers for climate-AI mixed precision
5. **Benchmarking:** Standard test cases and metrics
6. **Community building:** Training, workshops, collaboration

10. Conclusions

Mixed precision computing sits at the nexus of E3SM and AI, offering transformative potential for climate science:

For E3SM: - Enables exascale simulations through performance gains - Reduces energy consumption and computational cost - Allows higher resolution and longer simulations - Requires careful validation and algorithm adaptation

For AI in Climate: - Makes large-scale ML training and inference practical - Enables deployment of models within E3SM - Supports processing of massive climate datasets - Accelerates the AI-for-climate revolution

At the Nexus: - Hybrid models combining E3SM physics and ML data learning - Real-time climate prediction and digital twins - Foundation models for climate science - Democratized access to advanced climate modeling

The path forward requires collaboration between climate scientists, AI researchers, and computer scientists. Success demands: - Rigorous validation of mixed precision approaches - Development of robust software infrastructure - Training of next-generation researchers - Sustained investment in research and development

Mixed precision is not just an optimization technique—it's an enabler of the next generation of climate science, where exascale physics-based models and powerful AI systems work together to address humanity's most pressing challenge: understanding and predicting our changing climate.

11. Resources and Further Reading

E3SM Resources

- E3SM Project: <https://e3sm.org>
- E3SM Documentation: <https://docs.e3sm.org>
- E3SM GitHub: <https://github.com/E3SM-Project>

Kokkos and EAMxx Resources

- Kokkos GitHub: <https://github.com/kokkos/kokkos>
- Kokkos Tutorials: <https://github.com/kokkos/kokkos-tutorials>
- EAMxx/SCREAM: <https://github.com/E3SM-Project/E3SM> (components/eamxx)
- Trott et al., "Kokkos 3: Programming Model Extensions for the Exascale Era" (2022)
- Bertagna et al., "SCREAM: A Performance-Portable Atmosphere Model" (2023)

Mixed Precision Computing

- IEEE 754 Standard for Floating-Point Arithmetic
- NVIDIA Mixed Precision Training Guide
- Mixed Precision Training (Micikevicius et al., 2018)
- A Survey of Mixed Precision Techniques (Haidar et al., 2020)

AI for Climate

- ClimateBench: Climate model emulation benchmarks
- ClimateNet: Deep learning for climate pattern detection
- Haupt et al., "Towards physics-inspired AI for weather and climate" (2023)

Relevant Conferences

- SC (Supercomputing) - HPC focus
- NeurIPS - AI/ML focus
- AGU/AMS - Climate science focus
- AI4Science workshops

Software Tools

- PyTorch AMP
- TensorFlow mixed precision API
- NVIDIA Apex
- E3SM code repository
- Xarray for climate data processing

This primer was prepared for the E3SM and AI research community to facilitate understanding and adoption of mixed precision techniques at the intersection of climate modeling and artificial intelligence.

Version 1.1 - March 2026 (added Kokkos/EAMxx technical deep-dive)